Introduction to Processor Architecture (IPA) — Spring 2026
Mid-Semester Examination, IIIT HYDERABAD

Total Marks: 60	**Duration:** 90 Minutes
Course: IPA (CS 2.203)	**Date:** Spring 2026

GENERAL INSTRUCTIONS

1. This paper contains THREE questions. Answer ALL parts together and carefully.
2. Write all the answers in the answer-sheet provided. Calculator use is allowed.
4. Assume a standard 5-stage RISC-V pipeline: IF → ID → EX → MEM → WB, unless stated otherwise.
5. Registers are written in the first half and read in the second half of the clock cycle (write-before-read in same cycle).
6. Register x0 is always hardwired to zero in RISC-V.
7. Draw clear, fully labelled diagrams wherever required. Marks are allocated for clarity and correctness.
8. State any additional assumptions explicitly.

QUESTION 1: PIPELINED RISC-V DATAPATH & HAZARD HANDLING [25 MARKS]

Context:

You are designing a 5-stage in-order pipelined RISC-V processor that supports the RV32I base integer instruction set. The pipeline stages are: Instruction Fetch (IF), Instruction Decode/Register Read (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Pipeline registers IF/ID, ID/EX, EX/MEM, and MEM/WB separate the stages. The branch outcome and target address are resolved at the end of the EX stage. The processor uses a static branch prediction policy of Predict Not Taken (always fetches PC+4).

Consider the following RISC-V program fragment executing on this pipeline:

Label	Instruction
	I1: lw x1, 0(x10)
	I2: add x3, x1, x2
M/WB	I3: sw x3, 4(x10)
	I4: bne x3, x0, TARGET
	I5: addi x12, x10, 8
	I6: lw x13, 0(x12)
	...
TARGET:	I9: sub x8, x1, x3
	I10: and x9, x8, x2
	I11: sw x9, 12(x10)

Assume the branch in I4 is TAKEN (i.e., x3 ≠ 0 after the add).

(a) Data Dependency and Hazard Identification **[10 Marks]**

i. List ALL Read-After-Write (RAW) data dependencies in the instruction sequence I1 through I11. For each dependency, clearly state the producer instruction, the consumer instruction, and the register involved. Present your answer in a table template as below. [5 Marks]

Dependency #	Producer Instruction	Consumer Instruction	Register
1			
2			
...			

ii. Among the RAW dependencies identified above, classify each as one of: (A) resolvable by EX/MEM forwarding (MEM-to-EX bypass), (B) resolvable by MEM/WB forwarding (WB-to-EX bypass), or (C) a load-use hazard that requires at least one stall cycle even with full forwarding. Justify why load-use hazards cannot be resolved by forwarding alone. [5 Marks]

0.5 each (3)

2

(b) Complete Pipeline Execution Diagram **[15 Marks]**

Draw the complete multi-cycle pipeline execution diagram (clock-cycle chart – template shown below) for instructions I1 through I11, incorporating:

- Full data forwarding (EX/MEM and MEM/WB paths)
- One-cycle stall for load-use hazards
- Predict Not Taken with flush on misprediction (branch resolved in EX)
- The branch at I4 is TAKEN → execution continues at I9 (TARGET)

For each instruction, show the stage it occupies in each clock cycle. Clearly mark: stall bubbles, flushed instructions, and forwarding paths used (annotate with arrows or labels such as "Fwd EX/MEM"). Calculate the total number of clock cycles from the start of I1's IF to the completion of I11's WB.

1

***Hint:** Pay attention to the interaction between the load-use stall for I1→I2 and the subsequent forwarding and branch penalty. Count cycles carefully.*

Clock cycles

Instr	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1:lw	IF	ID	...	...										
I2:add		IF	...	...										
I3:sw								...						-
I4:bne														

I5:lw										...				
I6:add														
I9:sub													...	

[F] = Flushed (squashed due to branch misprediction), S = Stall, * = Forwarding used

QUESTION 2: CACHE MEMORY DESIGN & PERFORMANCE ANALYSIS [25 MARKS]

Context:

You are the memory subsystem architect for a single-core RISC-V processor (RV32I, 32-bit byte-addressable address space = 4 GB). The processor runs a 5-stage pipeline where the I-cache is accessed in the IF stage and the D-cache is accessed in the MEM stage. A cache miss stalls the pipeline until the data is available. Your task is to design and analyze the cache hierarchy.

(a) L1 Data Cache Design and Address Breakdown **[20 Marks]**

The L1 D-cache has the following specifications:

Parameter	Value
Cache Size	16 KB
Block (Line) Size	64 bytes
Associativity	4-way set-associative
Replacement Policy	LRU
Write Policy	Write-back, Write-allocate

i. Calculate: the total number of cache blocks, the number of sets, and the number of bits in each address field (offset, index, tag). Show all working. [5 Marks]

ii. Draw a fully labelled diagram of this cache organization showing: one complete set with all 4 ways, the tag array with valid and dirty bits, the comparators, the hit/miss detection logic (AND gates + OR gate), the data output multiplexer, and the LRU tracking mechanism. [10 Marks]

iii. Calculate the total storage overhead (in bits) for tag, valid, dirty, and LRU fields across the entire cache. Express this overhead as a percentage of the total cache storage (data + overhead). [5 Marks]

(b) Multi-Level Cache Performance (AMAT) **[5 Marks]**

The processor has the following memory hierarchy:

Level	Hit Time	Local Miss Rate	Size
L1 D-cache	1 cycle	6.25%	16 KB
L2 (unified)	10 cycles	15%	256 KB
Main Memory	100 cycles	—	512 MB

Compute the Average Memory Access Time (AMAT) using the recursive AMAT formula: AMAT = Hit_L1 + MissRate_L1 × (Hit_L2 + MissRate_L2 × Penalty_Mem). Show all steps.

2.5625 → BINARY (Unit Ute)

QUESTION 3: MULTIPLE CHOICE QUESTIONS ON CACHE MEMORY [10 MARKS]

Only one option correct. For each question, only write the correct choice label (A or B or C or D) in the answer sheet.

(a). A RISC-V program iterates through an array sequentially using a loop. The cache fetches an entire 64-byte block on the first access, and subsequent accesses to nearby array elements hit in the same block. Which type of locality does this cache behavior primarily exploit? [2 marks]

B

A) Temporal locality — because the same block is accessed repeatedly in the loop

B) Spatial locality — because nearby data items within the fetched block are accessed soon after

C) Instruction locality — because the loop instructions are stored close together in memory

D) Associative locality — because the block can be placed in any way of the cache set

(b). In the direct-mapped cache example from the lecture (4-line cache, block size = 1 word), the address sequence 0, 8, 0, 6, 8 results in a 0% hit rate. What is the primary cause of this behavior? [2 marks]

C

A) Compulsory misses — none of these blocks were ever cached before

B) Capacity misses — the working set exceeds the total cache size

C) Conflict misses — blocks 0 and 8 both map to the same cache line (line 0) and keep evicting each other

D) Coherence misses — another core invalidated the cache lines

(c). In a direct-mapped cache, two frequently accessed memory blocks happen to map to the same cache line. They continuously evict each other, resulting in a 0% hit rate for those blocks. This performance problem is known as thrashing. What is the most effective architectural solution to eliminate this specific problem? [2 marks]

C

A) Increase the cache block size so that both blocks fit within a single larger cache line

B) Switch to a write-back policy so that evictions do not propagate to main memory

C) Increase the cache associativity (e.g., use a 2-way set-associative cache) so both blocks can coexist in the same set

D) Use a random replacement policy instead of LRU to reduce the chance of repeated evictions

(d). Most modern RISC-V processor implementations, such as the SiFive U74, pair the write-back policy with the write-allocate miss policy for their L1 data cache. Which of the following best explains why this pairing is preferred over write-through with no-write-allocate? [2 marks]

B

A) Write-back keeps memory permanently consistent with the cache, which simplifies the coherence protocol

B) Write-back reduces memory bus traffic because writes are held in the cache and only written to memory on eviction, and write-allocate benefits from temporal locality on subsequent writes to the same block

C) Write-back eliminates the need for a dirty bit in each cache line, reducing hardware overhead

D) Write-through with no-write-allocate is incompatible with the RISC-V FENCE instruction and cannot be used

(e). A 32-bit RISC-V system has a direct-mapped cache with 128 entries and 32-byte blocks (as shown in the lecture). How many tag bits does each cache entry require? [2 marks]

C

A) 12 bits

B) 15 bits

C) 20 bits

D) 25 bits

——— END OF QUESTION PAPER ———